Project Title: Face Mask Detection Using Deep Learning

Tarek Rajab

Pandit Deendayal Energy University

Course Number: Artificial Intelligence

Abstract

This project aims to develop a deep learning system that can detect whether a person is wearing a face mask or not. The model was trained using image datasets and later integrated with a real-time camera system. Initially, a CNN model was built from scratch but later improved using transfer learning with MobileNetV2. After improving the dataset with more real-world cases, the final model achieved an accuracy of 98.9% and worked effectively in live detection.

Table of Contents

Introduction

During the COVID-19 pandemic, wearing face masks became necessary to prevent the spread of the virus. According to research, masks help reduce transmission of respiratory droplets, which are the main cause of infection spread.

However, manually checking whether people are wearing masks is difficult and not efficient. Therefore, artificial intelligence and deep learning can be used to automate this process. Face mask detection systems use image classification techniques to detect faces and classify them as mask or no mask in real time.

Problem Statement

The objective of this project is to:

- Detect faces in images or video
- Classify each face into:
    - Mask
    - No Mask
- Work in real-time using a webcam

Dataset

Two datasets from Kaggle were used:

Dataset 1: Face Mask Dataset

https://www.kaggle.com/datasets/omkargurav/face-mask-dataset

- Total images: 7553
- With mask: 3725
- Without mask: 3828
- RGB images organized in two folders

This dataset is commonly used for training mask detection models and provides balanced classes.

Dataset 2: Face Data for Mask Classification

https://www.kaggle.com/datasets/adsawe/face-data-for-mask-classification/data

- Contains images of:
    - Correct mask
    - Incorrect mask
    - No mask
- Includes real-world variations such as:
    - Masks worn incorrectly
    - Hands or objects covering face
    - Different lighting and angles

This dataset helps improve model performance in real-world scenarios by adding more diversity and edge cases.

Methodology

4.1 Data Preprocessing

- Rescaling images (0–1)
- Train-validation split: 80/20

4.2 Model 1 – CNN from Scratch

- Built using TensorFlow/Keras
- Layers:
    - Conv2D
    - MaxPooling
    - Dense
    - Dropout

Result:

- Accuracy ≈ 95%

Issue:

- Not very accurate in real-life detection
- Face detection performance was weak

4.3 Model 2 – Transfer Learning

Used pretrained model MobileNetV2

Steps:

- Load pretrained model (ImageNet weights)
- Freeze base layers

- Add custom layers
- Train on dataset

Result:

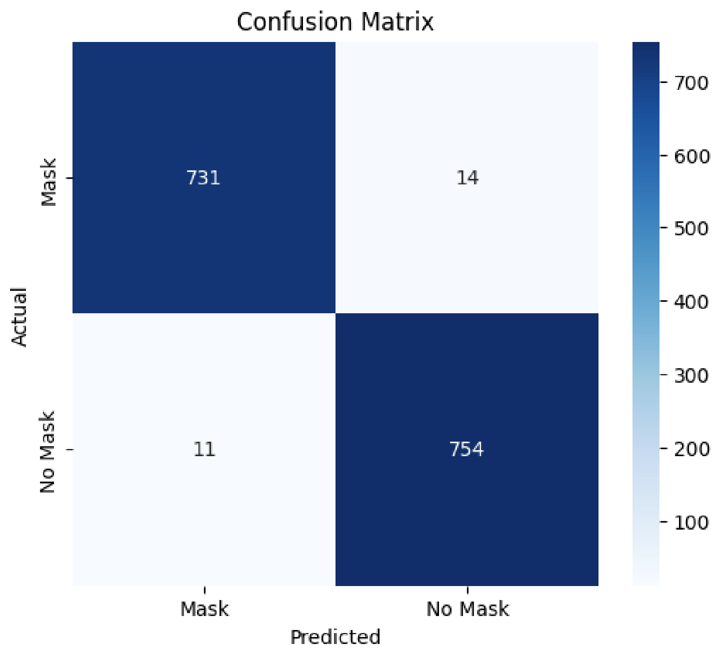
- Accuracy ≈ 98%
- Better performance than CNN

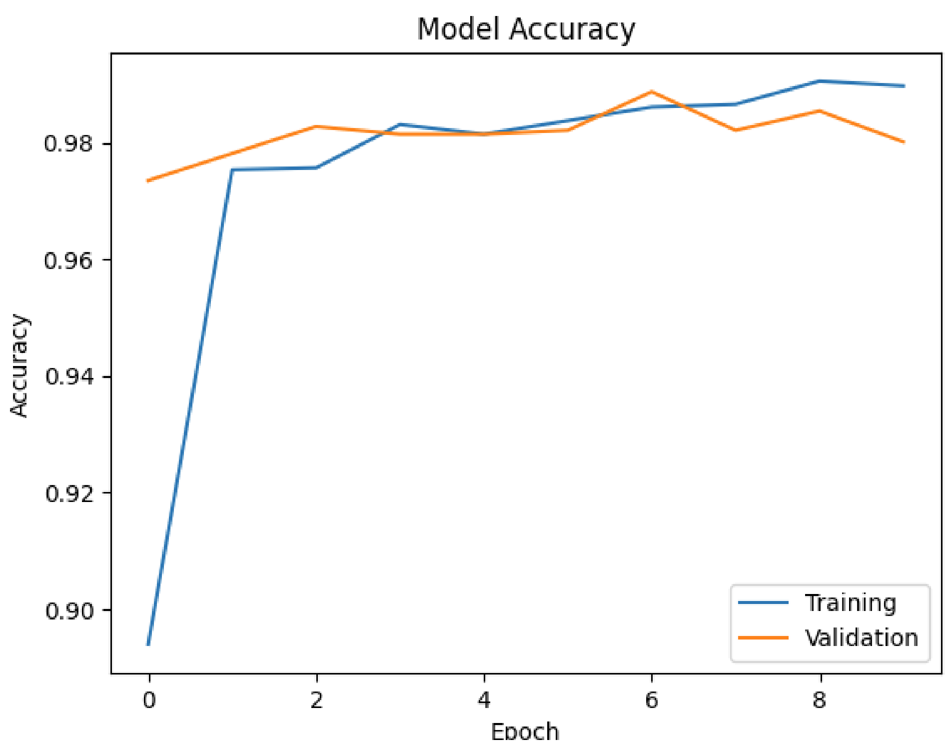
```
 Total params: 2,422,210 (9.24 MB)
 Trainable params: 164,226 (641.51 KB)
 Non-trainable params: 2,257,984 (8.61 MB)
Epoch 1/10
 50/189 ━━━━━━━━━━━━━━━━━━━━ 11:46 5s/step - accuracy: 0.6507 - loss: 0.7214/usr/local/lib/python3.12/d
  warnings.warn(
189/189 ━━━━━━━━━━━━━━━━━━━━ 1278s 6s/step - accuracy: 0.8939 - loss: 0.2528 - val_accuracy: 0.9735 - v
Epoch 2/10
189/189 ━━━━━━━━━━━━━━━━━━━━ 416s 2s/step - accuracy: 0.9753 - loss: 0.0811 - val_accuracy: 0.9781 - va
Epoch 3/10
189/189 ━━━━━━━━━━━━━━━━━━━━ 404s 2s/step - accuracy: 0.9757 - loss: 0.0682 - val_accuracy: 0.9828 - va
Epoch 4/10
189/189 ━━━━━━━━━━━━━━━━━━━━ 412s 2s/step - accuracy: 0.9831 - loss: 0.0501 - val_accuracy: 0.9815 - va
Epoch 5/10
189/189 ━━━━━━━━━━━━━━━━━━━━ 491s 3s/step - accuracy: 0.9815 - loss: 0.0478 - val_accuracy: 0.9815 - va
Epoch 6/10
189/189 ━━━━━━━━━━━━━━━━━━━━ 493s 3s/step - accuracy: 0.9838 - loss: 0.0465 - val_accuracy: 0.9821 - va
Epoch 7/10
189/189 ━━━━━━━━━━━━━━━━━━━━ 475s 3s/step - accuracy: 0.9861 - loss: 0.0402 - val_accuracy: 0.9887 - va
Epoch 8/10
189/189 ━━━━━━━━━━━━━━━━━━━━ 406s 2s/step - accuracy: 0.9866 - loss: 0.0405 - val_accuracy: 0.9821 - va
Epoch 9/10
189/189 ━━━━━━━━━━━━━━━━━━━━ 415s 2s/step - accuracy: 0.9906 - loss: 0.0303 - val_accuracy: 0.9854 - va
Epoch 10/10
189/189 ━━━━━━━━━━━━━━━━━━━━ 415s 2s/step - accuracy: 0.9897 - loss: 0.0316 - val_accuracy: 0.9801 - va
48/48 ━━━━━━━━━━━━━━━━━━━━ 81s 2s/step - accuracy: 0.9834 - loss: 0.0498
Test Accuracy: 0.9834437370300293
48/48 ━━━━━━━━━━━━━━━━━━━━ 83s 2s/step
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.98      0.98       745
           1       0.98      0.99      0.98       765

    accuracy                           0.98      1510
   macro avg       0.98      0.98      0.98      1510
weighted avg       0.98      0.98      0.98      1510
```





4.4 Final Improvement

Problem:

- Hand or objects on face were detected as “Mask”

Solution:

- Added dataset images with:
    - Hands on face
    - Incorrect mask usage
- Retrained model

Final Accuracy:

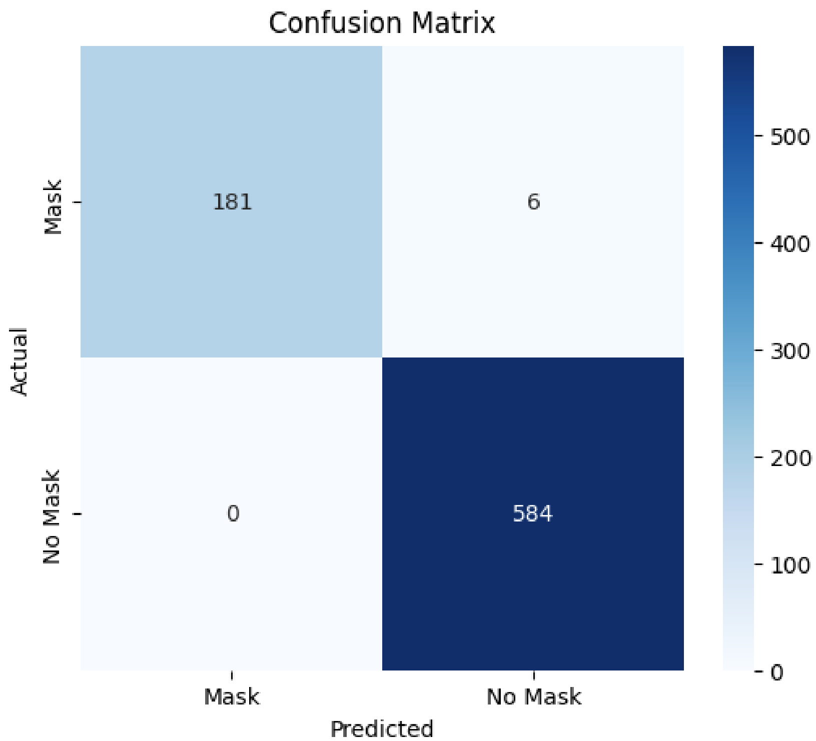
- 98.9%

```
 Total params: 2,422,210 (9.24 MB)
 Trainable params: 164,226 (641.51 KB)
 Non-trainable params: 2,257,984 (8.61 MB)
Epoch 1/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 0s 5s/step - accuracy: 0.7583 - loss: 0.9196/usr/local/lib/python3.12/dist-packages/PIL/Image.py:1047: UserWarning: Palette imag
  warnings.warn(
97/97 ━━━━━━━━━━━━━━━━━━━━ 604s 6s/step - accuracy: 0.7868 - loss: 0.7622 - val_accuracy: 0.9844 - val_loss: 0.0434
Epoch 2/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 257s 3s/step - accuracy: 0.8636 - loss: 0.3864 - val_accuracy: 0.9702 - val_loss: 0.0775
Epoch 3/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 247s 3s/step - accuracy: 0.8801 - loss: 0.3137 - val_accuracy: 0.9637 - val_loss: 0.1049
Epoch 4/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 255s 3s/step - accuracy: 0.9041 - loss: 0.2597 - val_accuracy: 0.9715 - val_loss: 0.0764
Epoch 5/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 243s 2s/step - accuracy: 0.9096 - loss: 0.2264 - val_accuracy: 0.9780 - val_loss: 0.0578
Epoch 6/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 245s 3s/step - accuracy: 0.9203 - loss: 0.2082 - val_accuracy: 0.9818 - val_loss: 0.0566
Epoch 7/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 250s 3s/step - accuracy: 0.9229 - loss: 0.1874 - val_accuracy: 0.9831 - val_loss: 0.0516
Epoch 8/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 252s 2s/step - accuracy: 0.9401 - loss: 0.1540 - val_accuracy: 0.9844 - val_loss: 0.0408
Epoch 9/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 280s 3s/step - accuracy: 0.9381 - loss: 0.1630 - val_accuracy: 0.9857 - val_loss: 0.0343
Epoch 10/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 249s 3s/step - accuracy: 0.9414 - loss: 0.1639 - val_accuracy: 0.9870 - val_loss: 0.0363
Epoch 11/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 274s 3s/step - accuracy: 0.9427 - loss: 0.1543 - val_accuracy: 0.9883 - val_loss: 0.0381
Epoch 12/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 249s 3s/step - accuracy: 0.9491 - loss: 0.1412 - val_accuracy: 0.9883 - val_loss: 0.0333
Epoch 13/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 251s 3s/step - accuracy: 0.9488 - loss: 0.1411 - val_accuracy: 0.9857 - val_loss: 0.0386
Epoch 14/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 260s 3s/step - accuracy: 0.9475 - loss: 0.1413 - val_accuracy: 0.9818 - val_loss: 0.0411
Epoch 15/15
97/97 ━━━━━━━━━━━━━━━━━━━━ 248s 3s/step - accuracy: 0.9534 - loss: 0.1348 - val_accuracy: 0.9922 - val_loss: 0.0290
25/25 ━━━━━━━━━━━━━━━━━━━━ 56s 2s/step - accuracy: 0.9896 - loss: 0.0290
Test Accuracy: 0.9896238446235657
25/25 ━━━━━━━━━━━━━━━━━━━━ 59s 2s/step
```

```
Classification Report:
tput actions
              precision    recall  f1-score   support

           0       1.00      0.97      0.98       187
           1       0.99      1.00      0.99       584

    accuracy                           0.99       771
   macro avg       0.99      0.98      0.99       771
weighted avg       0.99      0.99      0.99       771
```




```
Updated model saved successfully
```

Results

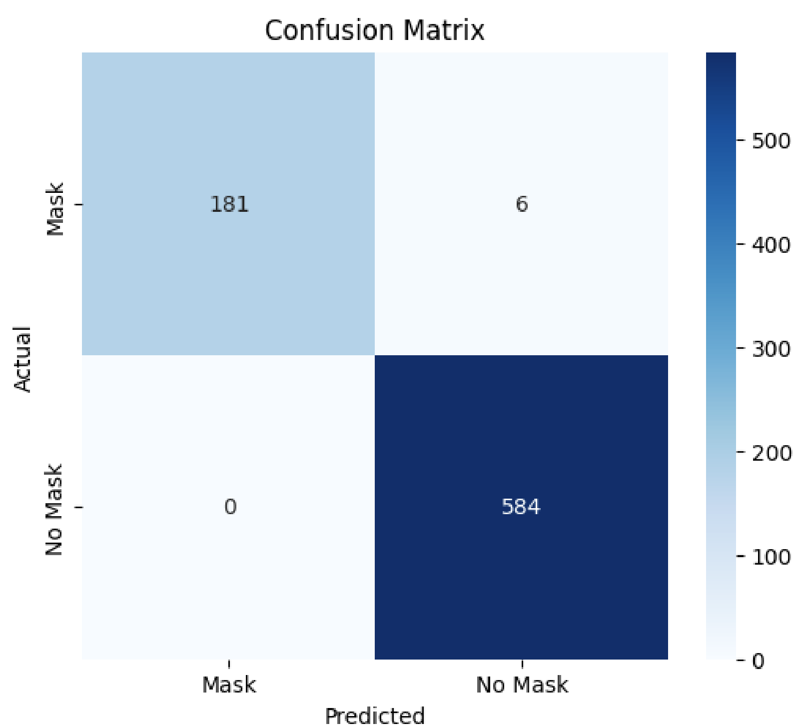
Model	Accuracy
CNN (Scratch)	95%
MobileNetV2	98%
New improved model	98.9%

The pretrained model performed better due to better feature extraction.

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.97      0.98       187
           1       0.99      1.00      0.99       584

    accuracy                           0.99       771
   macro avg       0.99      0.98      0.99       771
weighted avg       0.99      0.99      0.99       771
```




```
Updated model saved successfully
```

Application

Real-Time Application

The final model was implemented using:

- OpenCV
- Haar Cascade face detection
- Webcam

Working:

1. Capture video from camera
2. Detect faces
3. Predict mask/no mask
4. Display result

Challenges

- Face detection was not accurate initially
- Model confusing objects (hands) with masks
- Dataset lacked real-world diversity
- Needed optimization for real-time use

Conclusion

This project successfully created a real-time face mask detection system. Transfer learning improved accuracy and performance significantly. Adding more diverse dataset images solved misclassification issues and made the model more reliable.

References

1. **Face Mask Dataset (Kaggle)**
2. **Face Data for Mask Classification (Kaggle)**
3. **Real-time mask detection using deep learning and MobileNetV2**
4. **Research on mask detection and COVID-19 importance**